TinyGPU-ML

A Small GPU-like Accelerator for Machine Learning on NEORV32

Pre-planning Architectural Specification

Target platform:	NEORV32-based RISC-V SoC
Project type:	Hardware/software co-design accelerator project
Primary workload class:	Quantized inference kernels and small neural-network layers
Focus:	Architecture, implementation, integration, and evaluation plan

---

Contents

1 Executive Summary

The project implements a compact ML accelerator, called TinyGPU-ML, attached to a NEORV32 host processor. The accelerator executes a small set of machine-learning kernels with a command-driven programming model, local scratchpad memories, and a parallel compute array. The goal is to obtain a clear performance advantage over software execution on NEORV32 while preserving an implementation scope that is realistic for a semester project.

The architecture is organized around five core ideas:

- a **host-controlled execution model** in which NEORV32 launches kernels and manages memory,
- a **tiled dataflow** that reuses activation and weight tiles from local scratchpad,
- a $\mathbf{4 \times 4}$ **int8 MAC array** that computes one output tile at a time with int32 accumulation,
- an **epilogue stage** that applies bias, activation, and requantization before writeback,
- a **software-visible measurement interface** exposing cycles, stalls, and utilization.

The resulting system supports GEMM, GEMV, vector arithmetic, bias addition, ReLU/clamp, and conv2d execution through im2col plus GEMM. The primary numeric format is int8 input data with int32 accumulation and optional int8 output requantization.

2 Project Scope and Functional Targets

2.1 What "small GPU-like" means in this project

The term "small GPU-like" refers to a command-driven parallel accelerator that executes kernels on local tiles using many simple arithmetic units. It does not refer to a graphics pipeline or a full software stack such as CUDA or OpenCL. The design adopts selected GPU concepts that are appropriate for ML inference on an embedded SoC:

- parallel arithmetic lanes organized as a small compute array,
- explicit kernel launch from a host processor,
- local shared memory in the form of scratchpads,
- tiled execution and explicit data movement,
- simple occupancy and performance counters.

The design excludes graphics rendering, training workloads, floating-point-first execution, virtual memory, hardware cache coherence, and operating-system-managed device stacks.

2.2 Supported functionality

Kernel class	Operation	Purpose in the system
Tensor kernel	GEMM / GEMV	Main building block for dense layers and matrix operations.
Tensor kernel	Conv2D via im2col + GEMM	Supports convolutional layers without a separate dedicated convolution datapath.
Vector kernel	Vector add, vector multiply, scalar add/-multiply	Supports elementwise tensor operations and simple post-processing.
Epilogue kernel	Bias add, ReLU, clamp	Applies per-output-channel post-processing immediately after accumulation.
Quantization kernel	Requantize int32 to int8	Produces compact quantized outputs for downstream layers.
Reduction support	Partial sum / row sum	Used internally for accumulation and simple reduction-based post-processing.

Table 1: Supported functionality in the present project scope.

2.3 Numerical format

The accelerator uses quantized integer arithmetic:

- **Inputs and weights:** signed int8,
- **Accumulation:** signed int32,
- **Bias:** signed int32,
- **Output:** int32 or requantized int8.

This choice aligns with small ML inference workloads, reduces memory footprint, maps efficiently to FPGA DSP resources, and keeps the RTL manageable. All arithmetic units and memory interfaces are dimensioned around this format.

3 Architecture Overview

3.1 Top-level organization

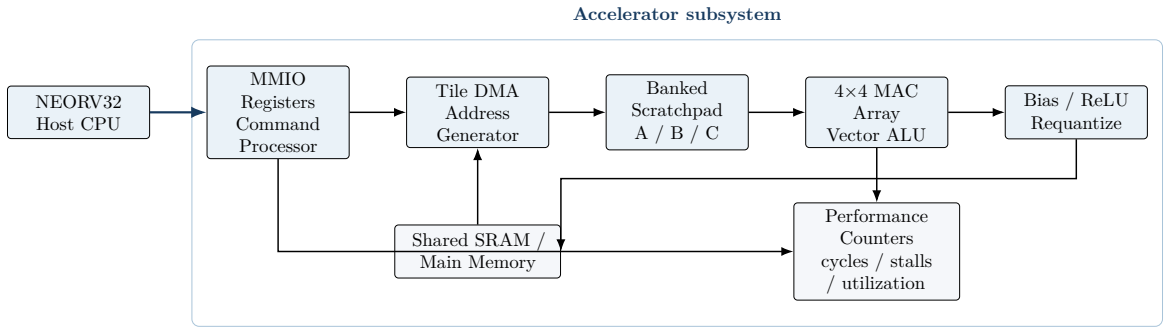

Figure 1: System dataflow and block-level organization of TinyGPU-ML. The NEORV32 host launches kernels through the MMIO command path, the DMA engine fetches tiles from shared memory into the banked scratchpad, the MAC array performs the core computation, the epilogue stage applies bias/activation/requantization, and the counter block exposes execution statistics.

3.2 Architectural intent

The host CPU is responsible for application control and kernel launch. The command processor decodes kernel descriptors and configures the DMA engine, scratchpad, and compute array. Tiles are moved from shared memory into local scratchpad, processed by the MAC array, passed through an epilogue stage for bias and activation, and then written back to memory. The performance-counter block measures execution cost without changing the programming model.

3.3 Block definitions

Block	Role in the system
NEORV32 host CPU	Runs the application, prepares descriptors, allocates input and output buffers, launches kernels, and checks completion and counters.
MMIO register bank	Exposes configuration and status through the SoC memory map. Supports both direct-launch mode and descriptor-based execution.
Command processor	Reads kernel parameters, validates opcode and dimensions, sequences the internal state machine, and coordinates launch, execution, and completion.
Tile DMA / address generator	Moves rectangular blocks between shared memory and local scratchpad. Generates burst addresses using base address, stride, element size, and tile shape.
Scratchpad memories	Store A-tiles, B-tiles, partial sums, and output tiles. Organized as independent banks to sustain one tile load and one compute access stream.
4×4 MAC array	Computes one 4×4 output tile. Each processing element performs int8 multiplication with int32 accumulation. Inputs are broadcast by rows and columns.
Vector ALU / epilogue stage	Applies bias, activation, clamp, and requantization to completed output values before writeback.

Performance counters	Count total cycles, active compute cycles, DMA cycles, and stall cycles caused by memory or control dependencies.

4 Detailed Microarchitecture

4.1 Execution model

Each kernel is executed as a sequence of tiles. For GEMM, the accelerator computes

$$C[M, N] = A[M, K] \times B[K, N]$$

by iterating over tiles of size

$$T_M = 4, \qquad T_N = 4, \qquad T_K = 16.$$

For one tile, the compute array accumulates a 4×4 block of C across a depth of 16 values from the K dimension. If the problem dimensions exceed one tile, the controller iterates over M, N, and K tile coordinates until the full output is completed.

This tile choice is concrete and implementable:

- a 4×4 output tile matches the 16 processing elements of the array,
- a K-depth of 16 allows reasonable data reuse while keeping scratchpad requirements low,
- edge tiles can be masked for dimensions that are not multiples of 4 or 16.

4.2 Compute array

The compute core is a **4×4 processing-element array**. Each processing element (PE) contains:

- one signed int8 multiplier,
- one int32 accumulator register,
- enable and clear logic for tile boundaries,
- bypass support for writeback into the epilogue path.

The dataflow is **output stationary**. A-tile values are broadcast horizontally across rows, B-tile values are broadcast vertically across columns, and each PE retains its partial sum for one output element. After all 16 K-steps for the tile are completed, the 16 accumulators hold the full 4×4 output tile.

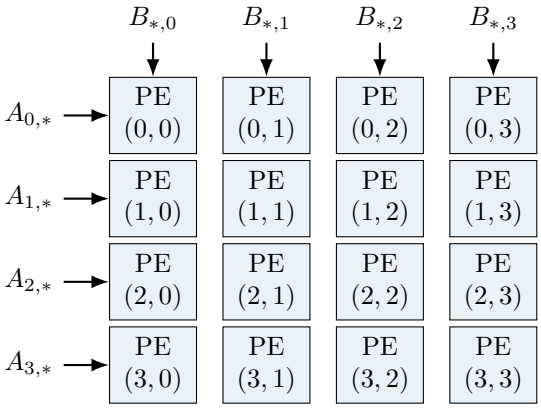

Figure 2: 4×4 output-stationary MAC array. Each PE accumulates one output element for the current tile.

4.3 Scratchpad organization

The local memory is split into independent banks to allow clean access patterns:

- **A scratchpad:** 2 banks × 256 bytes each,
- **B scratchpad:** 2 banks × 256 bytes each,
- **C/output staging buffer:** 256 bytes,
- **Accumulator file:** 16 entries × 32 bits.

The A and B scratchpads are double-buffered at the tile level. While one set of buffers feeds the compute array, the next A and B tiles can be loaded into the other set by the DMA engine. The controller toggles the active bank set at tile boundaries.

4.4 Tile DMA and address generation

The DMA/address generator accepts the following descriptor fields:

- base address,
- row stride in bytes,
- tile height,
- tile width,
- element size,
- bank destination.

For GEMM, the controller issues DMA requests in this order:

1. load A tile (m, k) into the selected A-bank,
2. load B tile (k, n) into the selected B-bank,
3. compute partial tile $C(m, n)$,
4. repeat for all K-tiles,
5. run epilogue on the completed output tile,

6. store the C tile to memory.

The DMA logic supports non-unit strides so that tensors stored in row-major memory can be read without software copying.

4.5 Epilogue stage

The epilogue stage processes a completed 4×4 output tile after accumulation. For each output element it performs, in order:

1. optional bias addition,
2. optional ReLU or clamp,
3. optional scaling, right shift, rounding, and zero-point addition,
4. saturation to int8 when the destination format is quantized.

This structure allows a single GEMM kernel launch to produce layer-ready output values without forcing the CPU to launch separate post-processing kernels.

4.6 Kernel encoding and control registers

Listing 1: Kernel descriptor used by the command processor.

```
typedef struct {
    uint32_t opcode;        // GEMM, GEMV, VEC_ADD, VEC_MUL, RELU, CONV2D_IM2COL_GEMM
    uint32_t flags;         // signed mode, dst format, epilogue enable bits
    uint32_t src0_addr;     // input activation or matrix A
    uint32_t src1_addr;     // weights or matrix B
    uint32_t bias_addr;     // optional bias vector
    uint32_t dst_addr;      // output tensor / matrix C
    uint16_t M, N, K;       // main dimensions
    uint16_t stride0;       // bytes between rows / tensor lines for src0
    uint16_t stride1;       // bytes between rows / tensor lines for src1
    uint16_t stride_dst;    // bytes between rows / tensor lines for dst
    int32_t  scale;         // requantization scale factor
    int16_t  shift;         // requantization right shift
    int16_t  zero_point;    // output zero point
} tinygpu_cmd_t;
```

Register	Access	Function
CTRL	R/W	Start, soft reset, direct mode enable, interrupt enable.
STATUS	R	Busy, done, illegal opcode, shape error, and memory error bits.
CMD_ADDR	R/W	Descriptor address for descriptor mode.
DIRECT_OP	R/W	Opcode for direct-launch mode used during bring-up.
ARG0..ARG7	R/W	Base addresses and dimensions in direct-launch mode.
CYCLE_COUNT	R	Total cycles consumed by the last kernel.
ACTIVE_COUNT	R	Cycles during which the compute array was active.
STALL_COUNT	R	Cycles lost to DMA wait, buffer conflict, or control hazards.
CMD_COUNT	R	Number of commands completed since reset.

Table 3: MMIO-visible control and measurement interface.

5 Kernel Mapping Strategy

5.1 GEMM and GEMV

GEMM maps directly onto the tiled MAC array. For each output tile $C(m, n)$, the controller iterates over the K-dimension in chunks of 16. Each chunk loads a 4×16 A tile and a 16×4 B tile into scratchpad. The compute array performs 16 outer-product accumulation steps, one for each K slice, and produces a final 4×4 tile.

GEMV is the same computation with $N = 1$ or $M = 1$. The controller masks inactive columns or rows but uses the same datapath.

5.2 Conv2D

Convolution is executed through **im2col + GEMM**. Software lowers the input tensor into a matrix of patches and arranges filter weights into a matrix of flattened kernels. The accelerator then executes GEMM on these matrices. This choice avoids a dedicated convolution-specific address generator and keeps the hardware focused on one efficient datapath.

5.3 Vector and epilogue kernels

Vector add, vector multiply, ReLU, and clamp are executed by the vector ALU path. These kernels read one vector tile into scratchpad, operate lane-wise, and write the result back. For GEMM output, the epilogue stage applies bias and activation immediately after accumulation, which reduces extra memory traffic.

6 Integration with NEORV32

6.1 SoC modifications

The project does not modify the NEORV32 ISA. The accelerator is integrated as a memory-mapped peripheral on the SoC interconnect. The required system-level modifications are:

- add the accelerator MMIO register block to the address map,
- connect the accelerator master port or DMA path to the shared SRAM interface,
- add an interrupt line for completion notification,
- provide a clock-enable/reset path consistent with the rest of the SoC,
- expose the accelerator in the platform header files and linker-visible memory map.

6.2 Software stack

The firmware side consists of:

- a low-level MMIO driver,
- a descriptor builder,
- kernel wrappers such as `tinygpu_gemm()`, `tinygpu_vec_add()`, and `tinygpu_relu()`,
- a reference library executing the same kernels in plain C on NEORV32,

- benchmark programs and unit tests.

Listing 2: Example host-side API.

```
void tinygpu_gemm(const int8_t *A, const int8_t *B, const int32_t *bias,
                  int8_t *C, uint16_t M, uint16_t N, uint16_t K,
                  int32_t scale, int16_t shift, int16_t zp);

void tinygpu_vec_add(const int32_t *x, const int32_t *y, int32_t *z,
                     uint32_t length);
```

7 Implementation Strategy

7.1 RTL module breakdown

Module	Implementation content
`tinygpu_regs`	MMIO register file, start/done handling, interrupt status, and counters.
`tinygpu_cmd`	Descriptor decode, legality checks, and high-level command sequencing.
`tinygpu_dma`	Tile loader/store unit with stride handling and bank selection.
`tinygpu_spm`	Banked scratchpad memory wrappers and double-buffer selection logic.
`tinygpu_pe`	One processing element: int8 multiplier, int32 adder, accumulator register.
`tinygpu_array4x4`	Interconnect and control for the 16-PE array.
`tinygpu_epilogue`	Bias, activation, requantization, and output packer.
`tinygpu_top`	Top-level integration with the SoC bus and memory interface.

7.2 Development sequence

The implementation proceeds in a controlled sequence:

1. build the MMIO register block and direct-launch path,
2. implement one PE and the 4×4 array with a testbench using local stimuli,
3. add scratchpad memories and tile load/store control,
4. connect DMA and verify tiled GEMM end-to-end,
5. add bias, activation, and requantization,
6. integrate the accelerator into the NEORV32 SoC,
7. complete the software driver and benchmark suite.

This ordering keeps the bring-up effort incremental and ensures that each layer of the design is verified before full-system integration.

8 Corner Cases and Design Constraints

8.1 Functional corner cases

The architecture must handle the following cases explicitly:

- matrix dimensions that are not multiples of 4 or 16,
- vector lengths that do not fill a whole tile,
- zero-sized dimensions or illegal descriptors,
- negative values and sign extension for int8 inputs,
- accumulator growth and saturation during requantization,
- bias vectors shorter than the full tensor width,
- non-unit row stride in memory.

Edge tiles are handled by lane and PE masks generated by the controller. Invalid descriptor values terminate the command and set a status bit without performing memory writes.

8.2 Microarchitectural pressure points

The principal implementation pressure points are:

- sustaining enough memory bandwidth to keep the compute array active,
- avoiding scratchpad bank conflicts during tile refill,
- aligning control timing across DMA, compute, and epilogue stages,
- balancing double-buffering complexity against schedule risk,
- meeting timing closure for the 16-PE array and epilogue path.

These constraints shape the chosen array size and tile dimensions. A 4×4 array is large enough to demonstrate acceleration but small enough to be integrated cleanly on a modest FPGA target.

9 Verification Plan

9.1 Verification levels

1. **Arithmetic unit verification:** verify a single PE for signed multiplication, accumulation reset, accumulation enable, and overflow behavior.
2. **Array verification:** verify the 4×4 array against a software model for full and partial tiles.
3. **Scratchpad and DMA verification:** verify bank select, load/store ordering, burst generation, and stride handling.
4. **Kernel-level verification:** verify GEMM, GEMV, vector add, ReLU, and requantization using randomized descriptors.
5. **SoC-level verification:** run NEORV32 firmware that launches kernels and compares hardware output against the C reference implementation.

9.2 Reference models and checking

Two reference models are used:

- a **C reference library** compiled into the firmware test program,
- a **Python or host-side golden model** used to generate deterministic test vectors for RTL simulation.

Every test compares output data and status flags against the golden result. For performance counters, the checks validate monotonicity, non-zero activity for active kernels, and reasonable relationships such as

$$\texttt{ACTIVE\_COUNT} \leq \texttt{CYCLE\_COUNT}.$$

9.3 Coverage-oriented test list

- 1×1, 2×2, and 4×4 tiles,
- non-square GEMM shapes such as 4×16 by 16×3,
- dimensions with remainders, such as $M = 7$, $N = 10$, $K = 19$,
- all-zero inputs, identity-like matrices, and random signed inputs,
- bias-enabled and bias-disabled kernels,
- ReLU-only and ReLU-plus-requantize output paths,
- illegal descriptor cases for robustness of status reporting.

10 Evaluation Plan

10.1 Metrics

The project evaluation reports the following metrics:

- kernel latency in cycles,
- speedup relative to the NEORV32 C implementation,
- MAC throughput in operations per cycle,
- utilization computed as `ACTIVE_COUNT/CYCLE_COUNT`,
- FPGA or synthesis resource usage: LUTs, FFs, BRAMs, and DSPs,
- maximum clock frequency after synthesis or implementation.

10.2 Benchmark set

The benchmark set is chosen to match the architecture:

- GEMM: 16×16, 32×32, and 64×64,
- GEMV: 32×64 and 64×64,
- conv2d via im2col: small input layers representative of TinyML workloads,

- vector kernels: lengths of 64, 128, 256, and 512 elements.

Each benchmark is executed in software and hardware using the same input data. The report should separate pure compute gains from total end-to-end runtime so that DMA overhead is visible.

10.3 Results to include in the final report

The final report should contain:

- the top-level architecture diagram and the PE-array dataflow,
- the register map and command format,
- the achieved resource and timing numbers,
- speedup plots versus matrix size or vector length,
- a short discussion of memory-bound versus compute-bound behavior,
- the main limitations of the first implementation.

11 Conclusion

This document defines a clear and implementable architecture for the revised project scope. The design keeps the original matrix-multiplication core idea, broadens it into a compact ML accelerator, and places it within a complete host-accelerator system. The chosen 4×4 output-stationary MAC array, banked scratchpad, descriptor-driven command path, and fused epilogue provide a coherent basis for implementation, integration, and evaluation. The resulting project remains focused enough for a course setting while being rich enough to demonstrate architectural design, hardware/software co-design, and quantitative accelerator evaluation.